

Elimination 3: Omitted Elements/ Arrows Score

To do this, we first try to obtain the whole list of elements that must be animated throughout the whole process, irrespective of the actual sequence. We can't get this from the XML, because not all elements in the XML may explicitly make it to the actual animated list.

We must use the Animation Sequence instead

We take the Animation Sequence, the original diagram code, and the diagram image and form an exhaustive list of elements that must get animated in any animation video. We pass the code so that we can ground these elements from their IDs to their actual textual labels.

We must first use this prompt

Element Checklist Obtaining Script

You are an expert Diagram Code Analyst **and** Visual Evaluator. Your objective **is** to process an animation sequence JSON, cross-reference it **with** the original diagram code **and** original image, **and** extract a comprehensive, human-readable checklist of every structural element (nodes, blocks, edges) that makes an appearance **in** the animation.

You will be provided **with**:

1. The Original Image of the diagram.
2. The Original Diagram Code (which contains the raw definitions **and** text labels).

3. The Animation Sequence JSON (which lists elements by their IDs across various timestamps).

*** EXTRACTION RULES ***

1. Node and Block Naming: To name a node or block, trace its ID from the animation JSON back to the original code and extract the text label directly associated with it.
2. Unlabeled Blocks/Rasters: If a block, raster image, or node is completely unlabeled in the code, look at the Original Image and assign it a concise, semantically meaningful name (e.g., "Blue Convolution Box", "Input Raster Image").
3. Composite Elements: For composite elements that contain sub-parts, you must only include the textual label of the **whole** parent element in the checklist. Do not clutter the checklist with the names of internal sub-components.
4. Edge/Arrow Formatting: For arrows, extract the resolved semantic names of both the source and target elements. Format the output strictly as: `from: "[Source Name]" to: "[Target Name]"`.
5. Exclude Standalone Text: Do not create a checklist category for raw textual labels. We only care if the structural components (blocks, nodes, arrows) are influenced by the style. Use the text labels **only** to name the blocks and nodes they belong to.

*** TASK ***

Iterate through EVERY timestamp in the Animation Sequence JSON. Accumulate all the resolved, human-readable labels for `nodes`, `blocks`, and `arrows`. Deduplicate the lists so each element only appears once.

*** OUTPUT REQUIREMENT ***

Output ONLY valid JSON matching the exact schema below. Do not output conversational text.

EXAMPLE LOGIC

If the JSON has a block `"G_box"`, and the code shows this box has the text label "Generator".

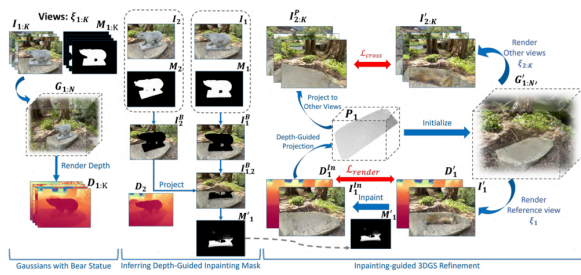
If the JSON has an arrow from `"I_stack_1"` (labeled "Input Images") to `"G_box"`.

Your output arrays will store: "Generator" in blocks, and "from: 'Input Images' to: 'Generator'" in edges.

```
```json
{
 "animation_checklist": {
 "blocks": [
 "Generator",
 "Projection Box P1",
 "Feature Extractor (Unlabeled Blue Box)"
],
 "nodes": [
 "Input Image Stack",
 "Depth Map D1",
 "Mask M1"
],
 "edges": [
 "from: 'Input Image Stack' to: 'Generator'",
 "from: 'Projection Box P1' to: 'Feature Extractor (Unlabeled Blue Box)'"
]
 }
}
```

## An example element checklist

```
{
 "animation_checklist": {
 "blocks": [
 "Gaussian Block $G_{1:N}$",
```



```

"Box I2, M2",
"Box I1, M1",
"Projection Box $P_1
$",
"Gaussian Block $G'_{1:N'}$"
],
"nodes": [
"Image Stack ($I_{1:K})$",
"Mask Stack ($M_{1:K})$",
"Depth Stack ($D_{1:K})$",
"Image I_2",
"Mask M_2",
"Image I_1",
"Mask M_1",
"Masked I_2",
"Masked I_1",
"Depth D_2",
"Projected I_2",
"Inpaint Mask $M'_1
$",
"Depth D'_1",
"Rendered I'_1",
"Composite Depth/Image D'_1, I'_1",
"Small Mask M'_1",
"Depth $D^{\{In\}}_1$",
"Inpainted $I^{\{In\}}_1
$",
"Composite Depth/Image $D^{\{In\}}_1, I^{\{In\}}_1$",
"Projection Stack $I^{\wedge}P_{2:K} $"

```

```

 "Render Stack $I'_
{2:K}$"
],
 "edges": [
 "from: 'Image Stack
($I_{1:K}$)' to: 'Gaussian
Block $G_{1:N}$'",
 "from: 'Gaussian Blo
ck $G_{1:N}$' to: 'Depth S
tack ($D_{1:K}$)'",
 "from: 'Mask M_2'
to: 'Masked I_2'",
 "from: 'Mask M_1'
to: 'Masked I_1'",
 "from: 'Masked $I_2
$' to: 'Depth D_2'",
 "from: 'Masked $I_1
$' to: 'Projected I_2'",
 "from: 'Depth D_2'
to: 'Projected I_2'",
 "from: 'Projected $I
_2$' to: 'Inpaint Mask
M'_1'",
 "from: 'Inpaint Mask
M'_1' to: 'Small Mask
M'_1'",
 "from: 'Rendered
I'_1' to: 'Inpainted $I^
{In}_1$'",
 "from: 'Small Mask
M'_1' to: 'Inpainted $I^
{In}_1$'",
 "from: 'Depth $D^{I
n}_1$' to: 'Projection Box
P_1'",
 "from: 'Projection B

```

```

ox P_1' to: 'Projection
Stack $I^P_{2:K}$',
 "from: 'Projection B
ox P_1' to: 'Gaussian Bl
ock $G'_{1:N}$'",
 "from: 'Gaussian Blo
ck $G'_{1:N}$' to: 'Rende
r Stack $I'_{2:K}$'",
 "from: 'Gaussian Blo
ck $G'_{1:N}$' to: 'Rende
red I'_1'",
 "from: 'Projection S
tack $I^P_{2:K}$' to: 'Ren
der Stack $I'_{2:K}$'",
 "from: 'Depth $D^{I
n}_1$' to: 'Depth D'_1'"
]
}
}

```

After this, we use the following prompts, once again with a generic instruction section and a style adaptation section, that help us detect omitted elements while adjusting to the peculiarities of every style

## Progressive Reveal

You are an expert Diagram Animation Omission Detector. Your objective **is** to track elements **from** a precomputed checklist **and** verify **if** they actually make an appearance **in** the animation sequence.

You are provided with:

1. The Original Diagram Image.
2. A sequence of Animation Frames in chronological order.
3. The Initial Animation Checklist containing "blocks", "nodes", and "edges".

### \*\*\* GENERIC INSTRUCTIONS \*\*\*

1. Checklist Popping: At each timestep (frame), observe which elements make an appearance. If an element appears, remove ("pop") it from your tracking list.
2. Semantic Mapping for Unlabeled Elements (CRITICAL): Some items in the checklist do not have explicit text labels in the image; they were given descriptive, semantic names (e.g., "Watermark Decoder" for an empty blue box). Do NOT assume an element is missing just because its text isn't written on the canvas. You MUST use the Original Diagram to logically map these semantic names to their corresponding visual shapes (using color, layout, context, and nearby arrows). Track the appearance of those specific shapes, not just text.
3. Lenient Text Matching: When matching elements that do have text, be highly lenient. Ignore exact math notations, LaTeX symbols (e.g.,  $I_{1:K}$ ), brackets, and minor text variations.
4. Final Pass: After evaluating the final frame, whatever remains in your tracked lists is considered an OMISSION.
5. Scoring:
  - `element\_omission\_count` = (Number of remaining blocks) + (Number of remaining nodes).
  - `arrow\_omission\_count` = Number of remaining edges.

### \*\*\* STYLE ADAPTER: PROGRESSIVE REVEAL \*\*\*

In Progressive Reveal, elements are cumulatively added to a blank or partially blank canvas.

- Appearance Rule: An element (node, block, or edge) is considered to have "appeared" and should be popped from the list the moment it is visually rendered on the canvas.

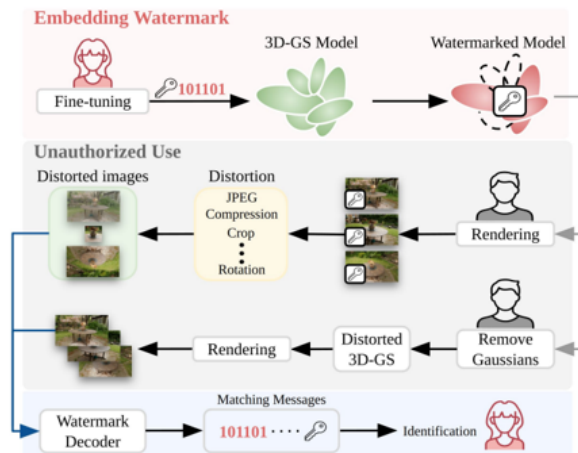
- You MUST track **and** pop blocks, nodes, AND edges.

\*\*\* OUTPUT REQUIREMENT \*\*\*

Output ONLY valid JSON matching the exact schema below.

```
```json
{
  "metadata": {
    "animation_style": "Progressive Reveal"
  },
  "timestep_reasoning": [
    {
      "frame_label": "string - frame filename or index",
      "reasoning": "Detailed rationale of what newly appeared. Explicitly state how you visually identified any unlabeled/semantically-named blocks.",
      "elements_popped_in_this_frame": {
        "blocks": ["List of block names popped"],
        "nodes": ["List of node names popped"],
        "edges": ["List of edge names popped"]
      }
    }
  ],
  "final_omissions": {
    "blocks_remaining": ["List of blocks that NEVER appeared"],
    "nodes_remaining": ["List of nodes that NEVER appeared"],
    "edges_remaining": ["List of edges that NEVER appeared"]
  },
  "element_omission_count": 0,
  "arrow_omission_count": 0
}
```


attachment:7106fa66-bc2d-4118-9954-569a418fd6d2:CVPR_2025_pipe00010.mp4



Original Diagram

Element Checklist for this Diagram

```
{
  "animation_checklist": {
    "blocks": [
      "Embedding Watermark",
      "Fine-tuning",
      "Unauthorized Use",
      "Rendering",
      "Distortion",
      "Distorted images",
      "Remove Gaussians",
      "Distorted 3D-GS",
      "Watermark Decoder B
```

```
{
  "metadata": {
    "animation_style": "Progressive Reveal"
  },
  "timestep_reasoning": [
    {
      "frame_label": "frame_1.png",
      "reasoning": "The frame is blank.",
      "elements_popped_in_this_frame": {
        "blocks": [],
        "nodes": [],
        "edges": []
      }
    },
    {
      "frame_label": "frame_2.png",
      "reasoning": "The top background block for embedding the watermark is introduced.",
      "elements_popped_in_this_frame": {
        "blocks": [
          "Embedding Watermark"
        ],
        "nodes": [],
        "edges": []
      }
    },
    {
      "frame_label": "frame_3.png",
      "reasoning": "The watermark is fully embedded and visible.",
      "elements_popped_in_this_frame": {
        "blocks": [
          "Embedding Watermark",
          "Fine-tuning",
          "Unauthorized Use",
          "Rendering",
          "Distortion",
          "Distorted images",
          "Remove Gaussians",
          "Distorted 3D-GS",
          "Watermark Decoder B"
        ],
        "nodes": [],
        "edges": []
      }
    }
  ]
}
```

```

lock",
    "Watermark Decoder",
    "Matching Messages"
  ],
  "nodes": [
    "Creator (Female)",
    "3D-GS Model",
    "Watermarked Model",
    "Unauthorized User
1",
    "Rendered Images",
    "Unauthorized User
2",
    "Distorted Images (Lower)"
  ],
  "edges": [
    "from: 'Fine-tuning'
to: '3D-GS Model'",
    "from: '3D-GS Model'
to: 'Watermarked Model'",
    "from: 'Watermarked
Model' to: 'Rendering'",
    "from: 'Watermarked
Model' to: 'Remove Gaussians'",
    "from: 'Rendering' to: 'Rendered Images'",
    "from: 'Rendered Images' to: 'Distortion'",
    "from: 'Distortion'
to: 'Distorted images'",
    "from: 'Remove Gaussians' to: 'Distorted 3D-GS'",
    "from: 'Distorted 3D

```

```

{
  "frame_label": "frame_3.png",
  "reasoning": "The initial watermarking process is shown, including the creator, the fine-tuning block, the original 3D-GS model, the watermarked model, and the connecting arrows.",
  "elements_popped_in_this_frame": {
    "blocks": [
      "Fine-tuning"
    ],
    "nodes": [
      "Creator (Female)",
      "3D-GS Model",
      "Watermarked Model"
    ],
    "edges": [
      "from: 'Fine-tuning' to: '3D-GS Model'",
      "from: '3D-GS Model' to: 'Watermarked Model'"
    ]
  }
},
{
  "frame_label": "frame_4.png",
  "reasoning": "The 'U

```

```

-GS' to: 'Rendering'",
    "from: 'Rendering' to: 'Distorted Images (Lower)'",
    "from: 'Distorted images' to: 'Watermark Decoder'",
    "from: 'Distorted Images (Lower)' to: 'Watermark Decoder'",
    "from: 'Watermark Decoder' to: 'Matching Messages'",
    "from: 'Matching Messages' to: 'Creator (Female)'"
  ]
}

```

```

authorized Use' section begins with the grey background block, a user icon, and the first 'Rendering' block, connected to the watermarked model.",
    "elements_popped_in_this_frame": {
      "blocks": [
        "Unauthorized Use",
        "Rendering"
      ],
      "nodes": [
        "Unauthorized User 1"
      ],
      "edges": [
        "from: 'Watermarked Model' to: 'Rendering'"
      ]
    },
    {
      "frame_label": "frame_5.png",
      "reasoning": "The rendering path for distorted images is revealed, including rendered images with key icons, the distortion block, the 'Distorted images' green block, and their respective arrows.",
      "elements_popped_in_

```

```

this_frame": {
    "blocks": [
        "Distortion",
        "Distorted images"
    ],
    "nodes": [
        "Rendered Images"
    ],
    "edges": [
        "from: 'Rendering' to: 'Rendered Images'",
        "from: 'Rendered Images' to: 'Distortion'",
        "from: 'Distortion' to: 'Distorted images'"
    ]
},
{
    "frame_label": "frame_6.png",
    "reasoning": "The second unauthorized use path is revealed: a second user, 'Remove Gaussians' block, 'Distorted 3D-GS' block, another 'Rendering' block, and the final set of 'Distorted Images (Lowe r)', with all connecting arrows.",
    "elements_popped_in_this_frame": {

```

```

        "blocks": [
            "Remove Gaussian
s",
            "Distorted 3D-G
S",
            "Rendering"
        ],
        "nodes": [
            "Unauthorized Us
er 2",
            "Distorted Image
s (Lower)"
        ],
        "edges": [
            "from: 'Watermar
ked Model' to: 'Remove Gau
ssians'",
            "from: 'Remove G
aussians' to: 'Distorted 3
D-GS'",
            "from: 'Distorte
d 3D-GS' to: 'Rendering'",
            "from: 'Renderin
g' to: 'Distorted Images
(Lower)'"
        ]
    }
},
{
    "frame_label": "fram
e_7.png",
    "reasoning": "The bo
ttom section for watermark
decoding is introduced, in
cluding the blue backgroun
d 'Watermark Decoder Bloc

```

```

k', the 'Watermark Decoder' block, and the blue arrows from both distorted image sets.",
    "elements_popped_in_this_frame": {
        "blocks": [
            "Watermark Decoder Block",
            "Watermark Decoder"
        ],
        "nodes": [],
        "edges": [
            "from: 'Distorted images' to: 'Watermark Decoder'",
            "from: 'Distorted Images (Lower)' to: 'Watermark Decoder'"
        ]
    },
    {
        "frame_label": "frame_8.png",
        "reasoning": "The final stage is revealed: the 'Matching Messages' block, the identification of the creator (female icon), and the final connecting arrows.",
        "elements_popped_in_this_frame": {
            "blocks": [

```

```

        "Matching Messages"
    ],
    "nodes": [],
    "edges": [
        "from: 'Watermark Decoder' to: 'Matching Messages'",
        "from: 'Matching Messages' to: 'Creator (Female)'"
    ]
}
},
{
    "final_omissions": {
        "blocks_remaining":
    [],
        "nodes_remaining": [],
        "edges_remaining": []
    },
    "element_omission_count": 0,
    "arrow_omission_count":
0
}

```

Alpha Masking

You are an expert Diagram Animation Omission Detector. Your objective is to track elements from a precomputed checklist and verify if they actually make an appearance in the animation sequence.

You are provided with:

1. The Original Diagram Image.
2. A sequence of Animation Frames in chronological order.
3. The Initial Animation Checklist containing "blocks", "nodes", and "edges".

*** GENERIC INSTRUCTIONS ***

1. Checklist Popping: At each timestep (frame), observe which elements make an appearance. If an element appears, remove ("pop") it from your tracking list.
2. Semantic Mapping for Unlabeled Elements (CRITICAL): Some items in the checklist do not have explicit text labels in the image; they were given descriptive, semantic names (e.g., "Watermark Decoder" for an empty blue box). Do NOT assume an element is missing just because its text isn't written on the canvas. You MUST use the Original Diagram to logically map these semantic names to their corresponding visual shapes (using color, layout, context, and nearby arrows). Track the appearance of those specific shapes, not just text.
3. Lenient Text Matching: When matching elements that do have text, be highly lenient. Ignore exact math notations, LaTeX symbols (e.g., $I_{1:K}$), brackets, and minor text variations.
4. Final Pass: After evaluating the final frame, whatever remains in your tracked lists is considered an OMISSION.
5. Scoring:
 - `element\_omission\_count` = (Number of remaining blocks) + (Number of remaining nodes).
 - `arrow\_omission\_count` = Number of remaining edges.

*** STYLE ADAPTER: ALPHA MASKING ***

In Alpha Masking, active elements are emphasized by fading in from transparent to fully opaque, while other elements are kept at a lower opacity.

- Appearance Rule: An element (node, block, or edge) is considered to have "appeared" and should be popped from the list the moment it becomes fully opaque/highlighted.
- You MUST track and pop blocks, nodes, AND edges.

*** OUTPUT REQUIREMENT ***

Output ONLY valid JSON matching the exact schema below.

```
```json
{
 "metadata": {
 "animation_style": "Alpha Masking"
 },
 "timestep_reasoning": [
 {
 "frame_label": "string - frame filename or index",
 "reasoning": "Detailed rationale of what newly became fully opaque. Explicitly state how you visually identified any unlabeled/semantically-named blocks.",
 "elements_popped_in_this_frame": {
 "blocks": ["List of block names popped"],
 "nodes": ["List of node names popped"],
 "edges": ["List of edge names popped"]
 }
 }
],
 "final_omissions": {
 "blocks_remaining": ["List of blocks that NEVER became fully opaque"],
 "nodes_remaining": ["List of nodes that NEVER became fully opaque"],
 "edges_remaining": ["List of edges that NEVER became full"]
 }
}
```

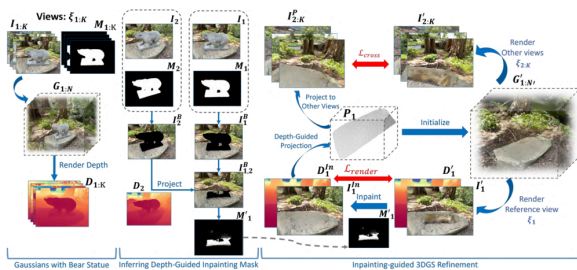
```

y opaque"]
},
"element_omission_count": 0,
"arrow_omission_count": 0
}

```

We get the following output for AnimateBanana video frames

attachment:85495e10-afbd-44e0-8e03-a7753e0972d7:CVPR\_2025\_pipe00004\_video\_tikz\_A  
M.mp4



Original Diagram

Element Checklist

```

{
 "animation_checklist": {
 "blocks": [
 "Gaussian Block $G_{1:N}$",

```

```

{
 "metadata": {
 "animation_style": "Alpha Masking",
 "timestep_reasoning": [
 {
 "frame_label": "frame_1.png",
 "reasoning": "The initial input images and masks are introduced.",
 "elements_popped_in_this_frame": {
 "blocks": [],
 "nodes": [
 "Image Stack ($I_{1:K}$)",
 "Mask Stack ($M_{1:K}$)"
],
 "edges": []
 }
 },
 {
 "frame_label": "frame_2.png",
 "reasoning": "The initial 3D Gaussian Splatting

```

```

 "Box I2, M2",
 "Box I1, M1",
 "Projection Box $P_1
$",
 "Gaussian Block $G'_
{1:N'}$"
],
 "nodes": [
 "Image Stack ($I_{1:
K}$)",
 "Mask Stack ($M_{1:
K}$)",
 "Depth Stack ($D_{1:
K}$)",
 "Image I_2",
 "Mask M_2",
 "Image I_1",
 "Mask M_1",
 "Masked I_2",
 "Masked I_1",
 "Depth D_2",
 "Projected I_2",
 "Inpaint Mask $M'_1
$",
 "Depth D'_1",
 "Rendered I'_1",
 "Composite Depth/Ima
ge D'_1, I'_1",
 "Small Mask M'_1",
 "Depth $D^{\{In\}}_1$",
 "Inpainted $I^{\{In\}}_1
$",
 "Composite Depth/Ima
ge $D^{\{In\}}_1, I^{\{In\}}_1$",
 "Projection Stack $I
^P_{2:K}$",

```

```

g model is introduced, alo
ng with the arrow connecti
ng the images to it.",
 "elements_popped_in_
this_frame": {
 "blocks": [
 "Gaussian Block
$G_{1:N}$"
],
 "nodes": [],
 "edges": [
 "from: 'Image St
ack ($I_{1:K}$)' to: 'Gaus
sian Block $G_{1:N}$'"
]
 },
 {
 "frame_label": "fram
e_3.png",
 "reasoning": "Depth
maps are rendered from the
pretrained model.",
 "elements_popped_in_
this_frame": {
 "blocks": [],
 "nodes": [
 "Depth Stack ($D
_{1:K}$)"
],
 "edges": [
 "from: 'Gaussian
Block $G_{1:N}$' to: 'Dept
h Stack ($D_{1:K}$)'"
]
 }
 }
]
}

```

```

 "Render Stack $I'_
{2:K}$"
],
 "edges": [
 "from: 'Image Stack
($I_{1:K}$)' to: 'Gaussian
Block $G_{1:N}$'",
 "from: 'Gaussian Blo
ck $G_{1:N}$' to: 'Depth S
tack ($D_{1:K}$)'",
 "from: 'Mask M_2'
to: 'Masked I_2'",
 "from: 'Mask M_1'
to: 'Masked I_1'",
 "from: 'Masked $I_2
$' to: 'Depth D_2'",
 "from: 'Masked $I_1
$' to: 'Projected I_2'",
 "from: 'Depth D_2'
to: 'Projected I_2'",
 "from: 'Projected $I
_2$' to: 'Inpaint Mask
M'_1'",
 "from: 'Inpaint Mask
M'_1' to: 'Small Mask
M'_1'",
 "from: 'Rendered
I'_1' to: 'Inpainted $I^
{In}_1$'",
 "from: 'Small Mask
M'_1' to: 'Inpainted $I^
{In}_1$'",
 "from: 'Depth $D^{I
n}_1$' to: 'Projection Box
P_1'",
 "from: 'Projection B

```

```

 },
 {
 "frame_label": "fram
e_4.png",
 "reasoning": "Indivi
dual reference views and t
arget masks are isolate
d.",
 "elements_popped_in_
this_frame": {
 "blocks": [
 "Box I2, M2",
 "Box I1, M1"
],
 "nodes": [
 "Image I_2",
 "Mask M_2",
 "Image I_1",
 "Mask M_1"
],
 "edges": []
 }
 },
 {
 "frame_label": "fram
e_5.png",
 "reasoning": "Backgr
ound regions are extracted
using the masks.",
 "elements_popped_in_
this_frame": {
 "blocks": [],
 "nodes": [
 "Masked I_2",
 "Masked I_1"
],

```

```

 ox P_1' to: 'Projection
Stack $I^P_{2:K}$',
 "from: 'Projection B
ox P_1' to: 'Gaussian Bl
ock $G'_{1:N}'$",
 "from: 'Gaussian Blo
ck $G'_{1:N}'$' to: 'Rende
r Stack $I'_{2:K}$'",
 "from: 'Gaussian Blo
ck $G'_{1:N}'$' to: 'Rende
red I'_1'",
 "from: 'Projection S
tack $I^P_{2:K}$' to: 'Ren
der Stack $I'_{2:K}$'",
 "from: 'Depth $D^{I
n}_1$' to: 'Depth D'_1'"
]
 }
}

```

```

 "edges": [
 "from: 'Mask $M_
2$' to: 'Masked I_2'",
 "from: 'Mask $M_
1$' to: 'Masked I_1'"
]
 }
},
{
 "frame_label": "fram
e_6.png",
 "reasoning": "Visibl
e background pixels are pr
ojected to find overlap
s.",
 "elements_popped_in_
this_frame": {
 "blocks": [],
 "nodes": [
 "Depth D_2",
 "Projected $I_2
$"
],
 "edges": [
 "from: 'Masked
I_2' to: 'Depth D_2'",
 "from: 'Masked
I_1' to: 'Projected $I_2
$'",
 "from: 'Depth $D
_2$' to: 'Projected $I_2
$'"
]
 }
},
{

```

```

 "frame_label": "frame_7.png",
 "reasoning": "The refined inpainting mask is constructed.",
 "elements_popped_in_this_frame": {
 "blocks": [],
 "nodes": [
 "Inpaint Mask M'_1"
],
 "edges": [
 "from: 'Projected I_2' to: 'Inpaint Mask M'_1'"
]
 },
 {
 "frame_label": "frame_8.png",
 "reasoning": "The refined mask is passed to the refinement stage.",
 "elements_popped_in_this_frame": {
 "blocks": [],
 "nodes": [
 "Small Mask M'_1"
],
 "edges": [
 "from: 'Inpaint Mask M'_1' to: 'Small Mask M'_1'"
]
 }
 }
]
}

```

```

]
 }
},
{
 "frame_label": "frame_9.png",
 "reasoning": "The reference view's rendered image and depth map are obtained.",
 "elements_popped_in_this_frame": {
 "blocks": [],
 "nodes": [
 "Depth D'_1 ",
 "Rendered I'_1 ",
 "Composite Depth/Image D'_1, I'_1 "
],
 "edges": []
 }
},
{
 "frame_label": "frame_10.png",
 "reasoning": "A 2D inpainting model is applied to the reference view.",
 "elements_popped_in_this_frame": {
 "blocks": [],
 "nodes": [
 "Depth $D^{\{In\}}_1$ ",
 "Inpainted $I^{\{I$

```

```

n}_1$",
 "Composite Depth/Image $D^{\text{In}}_1, I^{\text{In}}_1$"
],
 "edges": [
 "from: 'Rendered I'_1' to: 'Inpainted I^{In}_1'",
 "from: 'Small Mask M'_1' to: 'Inpainted I^{In}_1'"
]
 },
 {
 "frame_label": "frame_11.png",
 "reasoning": "A rendering loss is introduced between rendered and inpainted targets.",
 "elements_popped_in_this_frame": {
 "blocks": [],
 "nodes": [],
 "edges": [
 "from: 'Depth D^{In}_1' to: 'Depth D'_1'"
]
 }
 },
 {
 "frame_label": "frame_12.png",

```



```

 "reasoning": "The in
 painted region is projecte
 d into 3D space as anchor
 points.",
 "elements_popped_in_
 this_frame": {
 "blocks": [
 "Projection Box
 P_1"
],
 "nodes": [
 "Projection Stac
 k $I^P_{2:K}$"
],
 "edges": [
 "from: 'Depth $D
 ^{In}_1$' to: 'Projection
 Box P_1'",
 "from: 'Projecti
 on Box P_1' to: 'Project
 ion Stack $I^P_{2:K}$'"
]
 }
 },
 {
 "frame_label": "fram
 e_13.png",
 "reasoning": "New 3D
 Gaussians are initialized
 from anchor points.",
 "elements_popped_in_
 this_frame": {
 "blocks": [
 "Gaussian Block
 $G'_{1:N'}$"
],

```

```

 "nodes": [],
 "edges": [
 {
 "from": 'Projection Box P_1' to: 'Gaussian Block $G'_{1:N}'$"}
]
 },
 {
 "frame_label": "frame_14.png",
 "reasoning": "The updated model renders the scene from multiple views.",
 "elements_popped_in_this_frame": {
 "blocks": [],
 "nodes": [
 "Render Stack $I'_{2:K}$"
],
 "edges": [
 {
 "from": 'Gaussian Block $G'_{1:N}'$' to: 'Render Stack $I'_{2:K}'$',
 "from": 'Gaussian Block $G'_{1:N}'$' to: 'Rendered I'_1"}
]
 }
 },
 {
 "frame_label": "frame_15.png",
 "reasoning": "A cross-view consistency loss is

```

```

applied.",
 "elements_popped_in_
this_frame": {
 "blocks": [],
 "nodes": [],
 "edges": [
 "from: 'Projecti
on Stack $I^P_{2:K}$ ' to:
'Render Stack $I'_{2:K}$ '"
]
 }
},
],
"final_omissions": {
 "blocks_remaining":
[],
 "nodes_remaining": [],
 "edges_remaining": []
},
"element_omission_coun
t": 0,
"arrow_omission_count":
0
}

```

## Colour Pop

You are an expert Diagram Animation Omission Detector. Your objective **is** to track elements **from** a precomputed checklist **and** verify **if** they actually make an appearance **in** the animation sequence.

You are provided **with**:

1. The Original Diagram Image.
2. A sequence of Animation Frames in chronological order.
3. The Initial Animation Checklist containing "blocks", "nodes", and "edges".

\*\*\* GENERIC INSTRUCTIONS \*\*\*

1. Checklist Popping: At each timestep (frame), observe which elements make an appearance. If an element appears, remove ("pop") it from your tracking list.
2. Semantic Mapping for Unlabeled Elements (CRITICAL): Some items in the checklist do not have explicit text labels in the image; they were given descriptive, semantic names (e.g., "Watermark Decoder" for an empty blue box). Do NOT assume an element is missing just because its text isn't written on the canvas. You MUST use the Original Diagram to logically map these semantic names to their corresponding visual shapes (using color, layout, context, and nearby arrows). Track the appearance of those specific shapes, not just text.
3. Lenient Text Matching: When matching elements that do have text, be highly lenient. Ignore exact math notations, LaTeX symbols (e.g.,  $I_{1:K}$ ), brackets, and minor text variations.
4. Final Pass: After evaluating the final frame, whatever remains in your tracked lists is considered an OMISSION.
5. Scoring:
  - ``element_omission_count`` = (Number of remaining blocks) + (Number of remaining nodes).
  - ``arrow_omission_count`` = Number of remaining edges.

\*\*\* STYLE ADAPTER: COLOUR POP \*\*\*

In Colour Pop, the entire diagram starts in greyscale, and elements transition to full color when active.

- Appearance Rule: An element (node, block, or edge) is considered to have "appeared" and should be popped from the list the moment it transitions from greyscale to full color.
- You MUST track and pop blocks, nodes, AND edges.

\*\*\* OUTPUT REQUIREMENT \*\*\*

Output ONLY valid JSON matching the exact schema below.

```
```json
{
  "metadata": {
    "animation_style": "Colour Pop"
  },
  "timestep_reasoning": [
    {
      "frame_label": "string - frame filename or index",
      "reasoning": "Detailed rationale of what newly transitioned to color. Explicitly state how you visually identified any unlabeled/semantically-named blocks.",
      "elements_popped_in_this_frame": {
        "blocks": ["List of block names popped"],
        "nodes": ["List of node names popped"],
        "edges": ["List of edge names popped"]
      }
    }
  ],
  "final_omissions": {
    "blocks_remaining": ["List of blocks that NEVER transitioned to color"],
    "nodes_remaining": ["List of nodes that NEVER transitioned to color"],
    "edges_remaining": ["List of edges that NEVER transitioned to color"]
  },
  "element_omission_count": 0,
  "arrow_omission_count": 0
}
```

Hopping Bounding Box

You are an expert Diagram Animation Omission Detector. Your objective **is** to track elements **from** a precomputed checklist **and** verify **if** they actually make an appearance **in** the animation sequence.

You are provided **with**:

1. The Original Diagram Image.
2. A sequence of Animation Frames **in** chronological order.
3. The Initial Animation Checklist containing **"blocks"**, **"nodes"**, **and** **"edges"**.

*** GENERIC INSTRUCTIONS ***

1. Checklist Popping: At each timestep (frame), observe which elements make an appearance. If an element appears, remove (**"pop"**) it **from** your tracking **list**.
2. Semantic Mapping **for** Unlabeled Elements (CRITICAL): Some items **in** the checklist do **not** have explicit text labels **in** the image; they were given descriptive, semantic names (e.g., **"Watermark Decoder"** **for** an empty blue box). Do NOT assume an element **is** missing just because its text isn't written on the canvas. You MUST use the Original Diagram to logically **map** these semantic names to their corresponding visual shapes (using color, layout, context, **and** nearby arrows). Track the appearance of those specific shapes, **not** just text.
3. Lenient Text Matching: When matching elements that do have text, be highly lenient. Ignore exact math notations, LaTeX symbols (e.g., $I_{1:K}$), brackets, **and** minor text variations.
4. Final Pass: After evaluating the final frame, whatever remains **in** your tracked lists **is** considered an OMISSION.
5. Scoring:
 - ``element_omission_count`` = (Number of remaining blocks)

+ (Number of remaining nodes).

*** STYLE ADAPTER: HOPPING BOUNDING BOX ***

In Hopping Bounding Box, a transparent highlight box discretely jumps from one element to the next.

- Appearance Rule: An element (node or block) is considered to have "appeared" and should be popped from the list the moment the bounding box encloses it or highlights it.
- SPECIAL EDGE EXCEPTION: Bounding box animations DO NOT highlight arrows. You must COMPLETELY IGNORE the "edges" list. Do not attempt to pop edges.
- You must always output ``0`` for ``arrow_omission_count``.

*** OUTPUT REQUIREMENT ***

Output ONLY valid JSON matching the exact schema below.

```
```json
{
 "metadata": {
 "animation_style": "Hopping Bounding Box"
 },
 "timestep_reasoning": [
 {
 "frame_label": "string - frame filename or index",
 "reasoning": "Detailed rationale of what was enclosed by the bounding box. Explicitly state how you visually identified any unlabeled/semantically-named blocks.",
 "elements_popped_in_this_frame": {
 "blocks": ["List of block names popped"],
 "nodes": ["List of node names popped"]
 }
 }
],
 "final_omissions": {
 "blocks_remaining": ["List of blocks NEVER highlighted"],
 "nodes_remaining": ["List of nodes NEVER highlighted"]
 }
}
```

```

},
"element_omission_count": 0,
"arrow_omission_count": 0
}

```

## AnimateBanana Animation Video (TikZ)

[attachment:7c2c7abb-446d-4155-9631-829057f92872:CVPR\\_2025\\_pipe00011\\_video\\_tikz\\_HB.mp4](#)

Full pass can be seen at

<https://chat.qwen.ai/s/d1dc137c-5774-45e5-bf4c-09ac16677cdf?fev=0.2.84>

```

{
 "metadata": {
 "animation_style": "Hopping Bounding Box"
 },
 "timestep_reasoning": [
 {
 "frame_label": "frame_29",
 "reasoning": "The bounding box (red-dashed) enclosed the '$\\mathcal{L}_{total}$' node. Identified by the mathematical notation and position as the total loss component that combines all other loss terms in the optimization pipeline.",
 "elements_popped_in_this_frame": {
 "blocks": [],
 "nodes": ["$\\mathcal{L}_{total}$"]
 }
 },
 {
 "frame_label": "frame_30",
 "reasoning": "The bo

```



```

unding box (red-dashed) enclosed the '$\mathcal{L}_w$' node. Identified by the mathematical notation and position as the wavelet loss component calculated on high-frequency subbands.",
 "elements_popped_in_this_frame": {
 "blocks": [],
 "nodes": ["$\mathcal{L}_w$"]
 },
 {
 "frame_label": "frame_31",
 "reasoning": "The bounding box (red-dashed) enclosed the '$\mathcal{L}_{\text{rec}}$, $\mathcal{L}_{\text{lpips}}$' node. Identified by the mathematical notation and position as the reconstruction and perceptual loss components calculated between rendered and ground-truth images.",
 "elements_popped_in_this_frame": {
 "blocks": [],
 "nodes": ["$\mathcal{L}_{\text{rec}}$, $\mathcal{L}_{\text{lpips}}$"]
 }
 }

```

```

 },
 {
 "frame_label": "frame_32",
 "reasoning": "The bounding box (red-dashed) enclosed the 'High Frequency' node in the ground-truth processing path. Identified by the text label and position as the high-frequency components of the ground-truth image, matching 'High Frequency (GT)' in the checklist.",
 "elements_popped_in_this_frame": {
 "blocks": [],
 "nodes": ["High Frequency (GT)"]
 }
 },
 {
 "frame_label": "frame_33",
 "reasoning": "The bounding box (red-dashed) enclosed the Hadamard Product symbol ($\odot$) in the top process flow. Identified by the mathematical symbol and context as the operation combining gradient mask with other parameters during optimization.",
 "elements_popped_in_

```

```

this_frame": {
 "blocks": [],
 "nodes": ["Hadamard Product ($\odot$)"]
}
],
"final_omissions": {
 "blocks_remaining":
[],
 "nodes_remaining": []
},
"element_omission_count": 0,
"arrow_omission_count":
0
}

```

## Sliding Bounding Box

You are an expert Diagram Animation Omission Detector. Your objective **is** to track elements **from** a precomputed checklist **and** verify **if** they actually make an appearance **in** the animation sequence.

You are provided **with**:

1. The Original Diagram Image.
2. A sequence of Animation Frames **in** chronological order.
3. The Initial Animation Checklist containing **"blocks"**, **"nodes"**, **and** **"edges"**.

**\*\*\* GENERIC INSTRUCTIONS \*\*\***

1. Checklist Popping: At each timestep (frame), observe which

elements make an appearance. If an element appears, remove ("pop") it from your tracking list.

2. Semantic Mapping for Unlabeled Elements (CRITICAL): Some items in the checklist do not have explicit text labels in the image; they were given descriptive, semantic names (e.g., "Watermark Decoder" for an empty blue box). Do NOT assume an element is missing just because its text isn't written on the canvas. You MUST use the Original Diagram to logically map these semantic names to their corresponding visual shapes (using color, layout, context, and nearby arrows). Track the appearance of those specific shapes, not just text.

3. Lenient Text Matching: When matching elements that do have text, be highly lenient. Ignore exact math notations, LaTeX symbols (e.g.,  $I_{1:K}$ ), brackets, and minor text variations.

4. Final Pass: After evaluating the final frame, whatever remains in your tracked lists is considered an OMISSION.

5. Scoring:

- `element\_omission\_count` = (Number of remaining blocks)  
+ (Number of remaining nodes).

### \*\*\* STYLE ADAPTER: SLIDING BOUNDING BOX \*\*\*

In Sliding Bounding Box, a transparent highlight box smoothly slides across the canvas from one element to the next.

- Appearance Rule: An element (node or block) is considered to have "appeared" and should be popped from the list the moment the bounding box comes to rest and encloses/highlights it.

- SPECIAL EDGE EXCEPTION: Bounding box animations DO NOT highlight arrows. You must COMPLETELY IGNORE the "edges" list. Do not attempt to pop edges.

- You must always output `0` for `arrow\_omission\_count`.

### \*\*\* OUTPUT REQUIREMENT \*\*\*

Output ONLY valid JSON matching the exact schema below.

```
```json
```

```

{
  "metadata": {
    "animation_style": "Sliding Bounding Box"
  },
  "timestep_reasoning": [
    {
      "frame_label": "string - frame filename or index",
      "reasoning": "Detailed rationale of what was enclosed by the bounding box. Explicitly state how you visually identified any unlabeled/semantically-named blocks.",
      "elements_popped_in_this_frame": {
        "blocks": ["List of block names popped"],
        "nodes": ["List of node names popped"]
      }
    }
  ],
  "final_omissions": {
    "blocks_remaining": ["List of blocks NEVER highlighted"],
    "nodes_remaining": ["List of nodes NEVER highlighted"]
  },
  "element_omission_count": 0,
  "arrow_omission_count": 0
}

```

Note: Sliding bounding box will be pretty similar to Hopping bounding box, with just the difference of more frames involved in the slide. Since the animations at this stage have already passed the style check, we know for sure that the sliding bounding box is working really well. As a result, sample after every 4 frames in TikZ to save on the input frames per sample